Reelo AI PoC: Project Explanation

A tool-first AI layer over a simulated Odoo dataset. What I built, how each file works, and how to explain it.

1. What this project is

This is a small proof of concept. A user types a normal business question, like "Give me hot picks for CA under $5000" or "How much stock does SKU-1001 have?". The system works out what they want, runs plain Python tools to get the real facts, checks that the user is allowed to see them, and sends back a clear answer.

The data is not real Odoo. It is a JSON file (data/seed_data.json) that pretends to be Odoo: 60 products, 120 inventory rows and a few knowledge documents. I kept the tool contracts simple on purpose, so each tool can later be swapped for a real Odoo call without touching the rest of the code.

The whole thing is Python 3.10+ and the standard library only. There is nothing to install.

2. The one idea behind everything

Tools decide. The LLM only formats.

The business sells nicotine, THC and kratom products. If the system tells a salesperson a product is legal in a state when it is not, that is a real legal problem, not just a wrong answer. An LLM can sound very sure and still make up stock numbers or legality. So every fact in this project comes from a normal Python function that gives the same answer every time for the same input. If an LLM is added later, it can only rephrase a small, cleaned-up tool result. It never gets to decide stock, price or legality.

Every request is also logged, so afterwards I can show what ran, who asked, how long it took and whether it was allowed.

3. What the assignment asked, and where I did it

Requirement	Where it is done
Classify 5 intents without an LLM	app/routing/router.py (regex rules)
5 deterministic tools over the seed data	app/execution/tools.py
Chain A (sales), B (compliance), C (vendor)	app/execution/chains.py
Small session memory (4 fields)	app/state/state.py
Logs with request_id, user_type, intent, tools, latency, tokens	app/observability/observability.py
Tool allowlist by user type	app/safety/policy.py, engine.py, runner.py
PII redaction	app/safety/pii.py
Audit of tool calls	logs/audit.jsonl
Never put the dataset in a prompt	formatter.llm_prompt() cuts it at 4000 characters
Blocked items are never recommended	compliance_filter + validator.py

4. How one request flows

Every request goes through app/engine.py in the same fixed order. There are seven steps. The picture below is the whole project on one page, and the rest of this document zooms into each box.

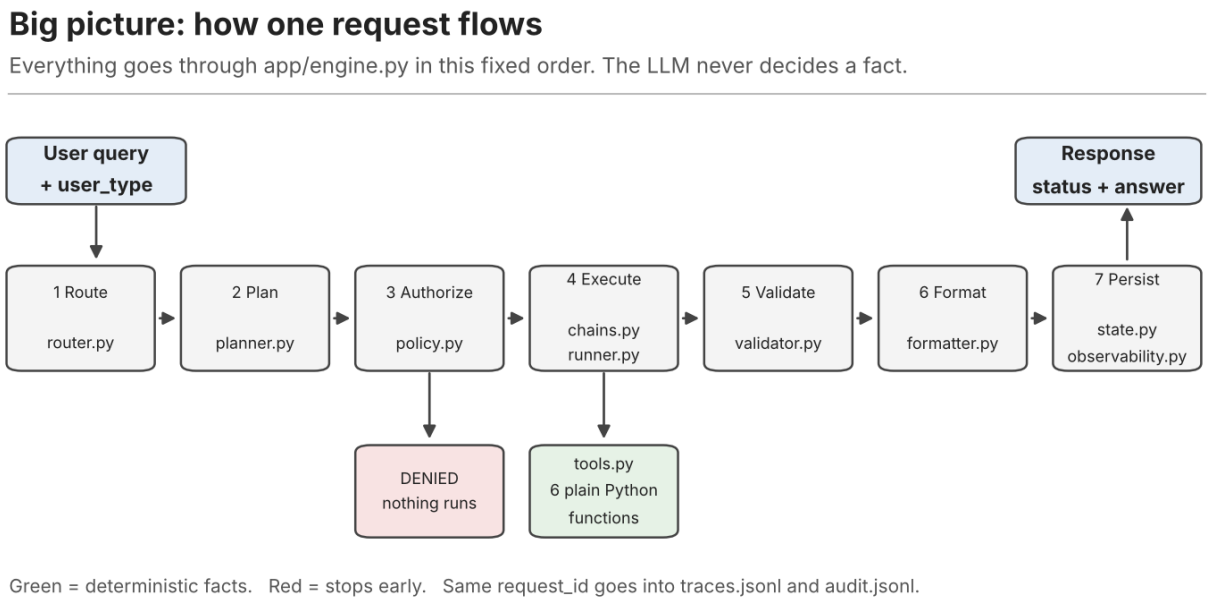


Figure 1. The 7 steps. Green is where facts come from, red is where a request stops early.

#	Step	What happens	File
1	Route	Regex picks one intent and a lane (TRUTH or KNOWLEDGE).	router.py
2	Plan	A fixed table gives the list of tools for that intent.	planner.py
3	Authorize	Every tool in the plan must be allowed for this user type. If not, the answer is DENIED and nothing runs.	policy.py
4	Execute	The chain runs. Each tool call goes through ToolRunner.	chains.py, runner.py
5	Validate	Checks that every item we are about to return had an ALLOWED verdict.	validator.py
6	Format	A template builds the text and estimates the prompt size.	formatter.py
7	Persist	Saves memory (only if status is OK) and writes the trace line.	state.py, observability.py

Two lanes

- **Truth lane:** stock, legality, price. These answers come only from tools.
- **Knowledge lane:** policy and SOP questions. A simple keyword search over 3 short documents.

I never use embeddings or RAG for inventory or compliance. Live facts and legal decisions only go through the truth lane.

5. Folder layout

```
reelo_ai_poc/
  main.py                entry point (demo or chat)
  README.md
  data/seed_data.json    products, inventory, customers, vendors, kb_docs
  docs/                  architecture.png, TECHNICAL_NOTE.md
  logs/                  traces.jsonl, audit.jsonl (created when you run it)
  app/
    engine.py            the 7-step pipeline
    routing/             router.py, planner.py, extract.py
    execution/           chains.py, runner.py, tools.py
    safety/              policy.py, validator.py, pii.py
    state/               state.py
    output/              formatter.py
    observability/       observability.py
```

Each folder answers one question:

Folder	The question it answers
routing	What does the user want, and which tools does that need?
execution	How do we get the facts?
safety	Is this allowed, and is the result safe to return?
state	What do we remember between messages?
output	How do we present the result?
observability	What happened, and how long did it take?

6. The data

The tools load data/seed_data.json once into memory. You can swap the file, or point the SEED_PATH environment variable at another one, with no code change.

Section	Rows	Used by
products	60	hot_picks, compliance_filter, stock, chains
inventory	120	stock_by_warehouse
customers	2	present in the file, not used by the 5 tools
vendors	2	present in the file, not used by the 5 tools
kb_docs	3	kb_search: Shipping Policy (public), Returns SOP (internal), Vendor Catalog Upload Guide (vendor)

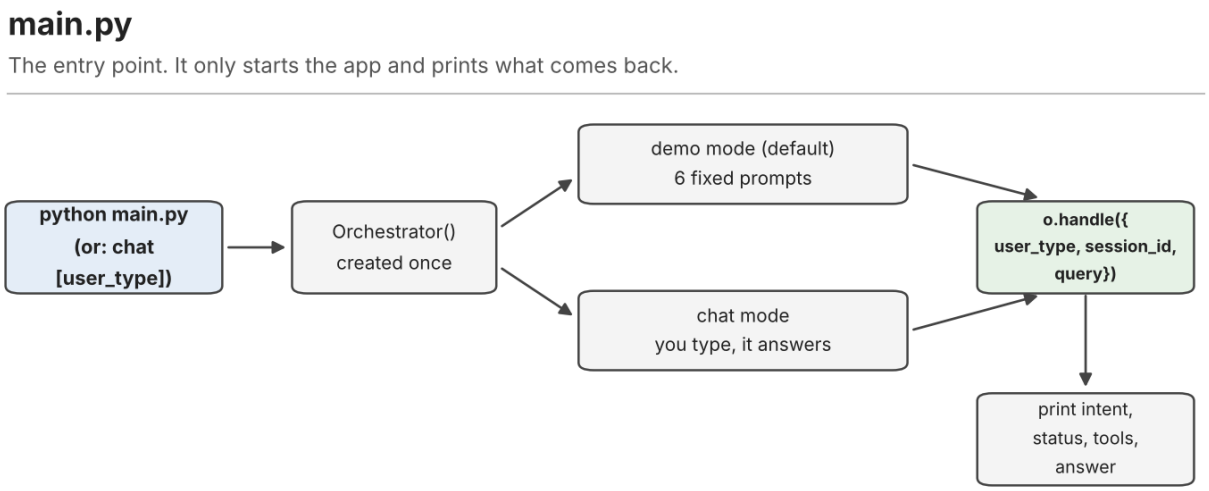
The 60 products are spread over six categories: Mushroom Chocolate (13), THC Beverage (11), Nicotine Vape (11), Kratom Capsules (10), Accessories (8) and CBD Gummies (7). Each product has a price, a popularity score, a list of blocked states and a flag for whether a lab report is required.

The system only answers questions in its own domain. A joke or a weather question goes to GENERAL_KB and comes back as "No matching document found". That is on purpose. This is a tool orchestrator, not a general chatbot.

7. File by file

Below, every Python file gets one page-sized section: a picture, then what it does, how it works and why I wrote it that way. They follow the order a request travels, starting from main.py.

main.py



The 6 demo prompts: hot picks, basket follow-up, compliance check, stock, vendor upload, and a permission denial.

What it does

This is the only file you run. It creates one Orchestrator and then either plays 6 ready-made prompts (demo mode) or lets you type your own (chat mode).

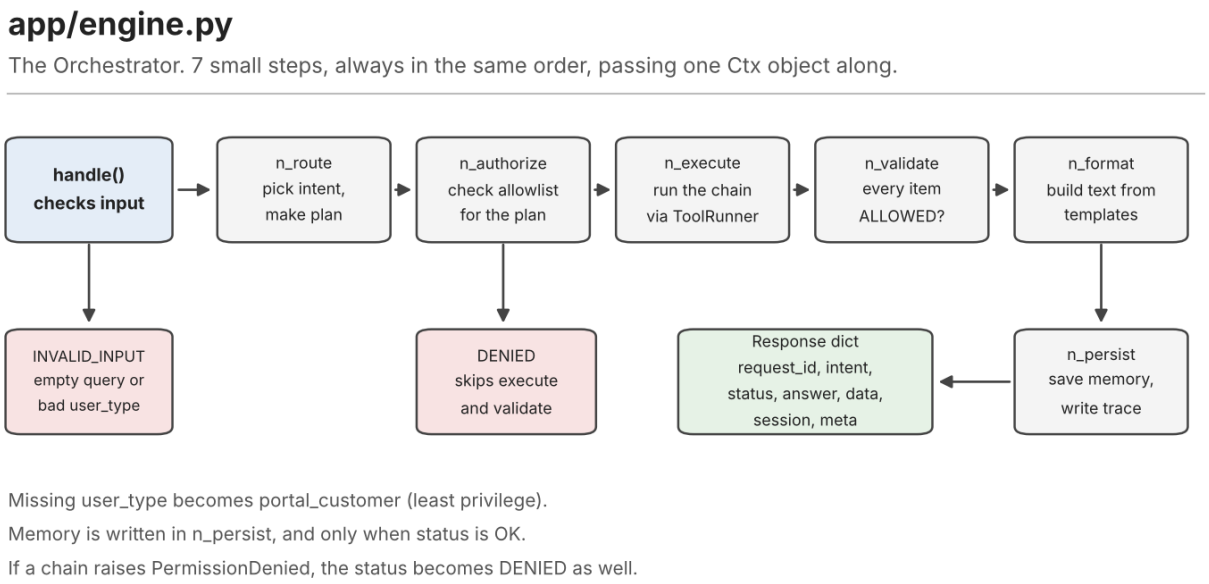
How it works

- `python main.py` runs the demo. `python main.py chat portal_vendor` starts chat as that user type. The default chat user is internal_sales.
- The demo picks a blocked product and an in-stock product from the data itself, so it still works if the seed file changes.
- The 6 prompts cover sales, a memory follow-up, compliance, stock, vendor upload and a permission denial.
- For each answer it prints the intent, the status, the tools that were called and the answer text.

Why I did it this way

I wanted one obvious place to start. A reviewer can run one command and see every important behaviour in about a second, without reading any code first.

app/engine.py



What it does

The Orchestrator. It owns the pipeline and calls the steps in a fixed order. It does no business logic itself.

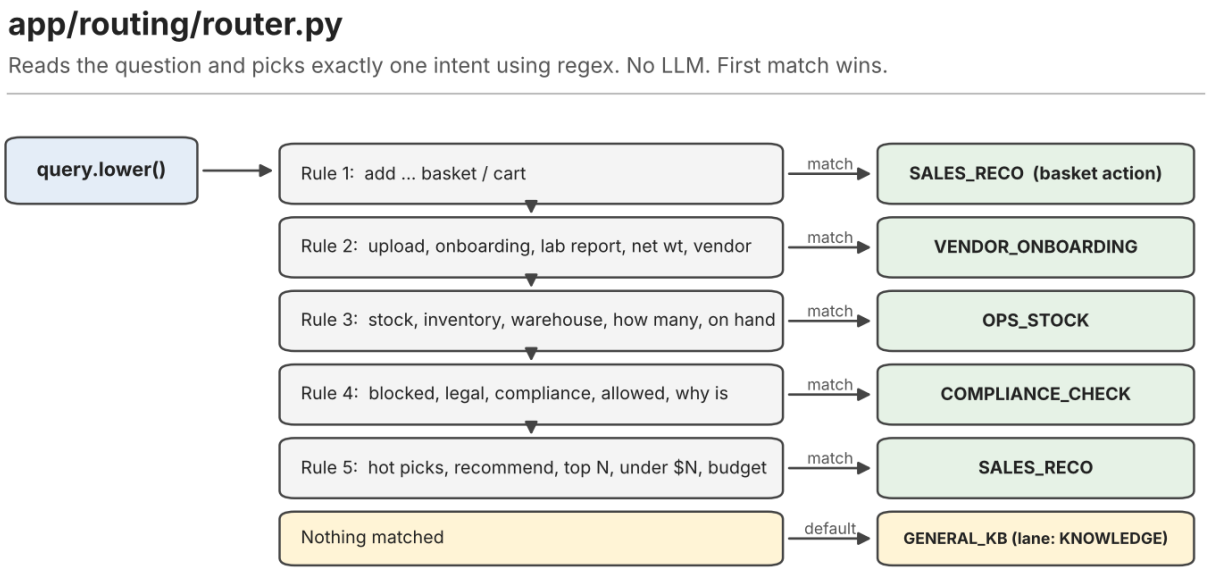
How it works

- handle() first checks the input. An empty query or an unknown user_type returns INVALID_INPUT straight away. A missing user_type becomes portal_customer.
- It then builds one Ctx object (request, user type, trace, session, route, plan, status, payload and so on) and passes it through the steps: n_route, n_authorize, n_execute, n_validate, n_format, n_persist.
- n_authorize checks the *whole plan* first. If any tool is not allowed, the status becomes DENIED and execute and validate are skipped. Format and persist still run, so even a denied request is logged.
- n_validate replaces a bad result with "Internal consistency check failed; result withheld." and clears the memory updates.
- n_persist saves memory only when the status is OK, writes the trace line and builds the final response dictionary.

Why I did it this way

Seven small steps are easier to read, test and explain than one big function. Because the order never changes, anyone can follow a request from top to bottom. And since the permission check happens before any tool runs, a forbidden request cannot have side effects.

app/routing/router.py



What it does

Reads the question and decides what the user wants. There are five intents plus a default. No LLM is used.

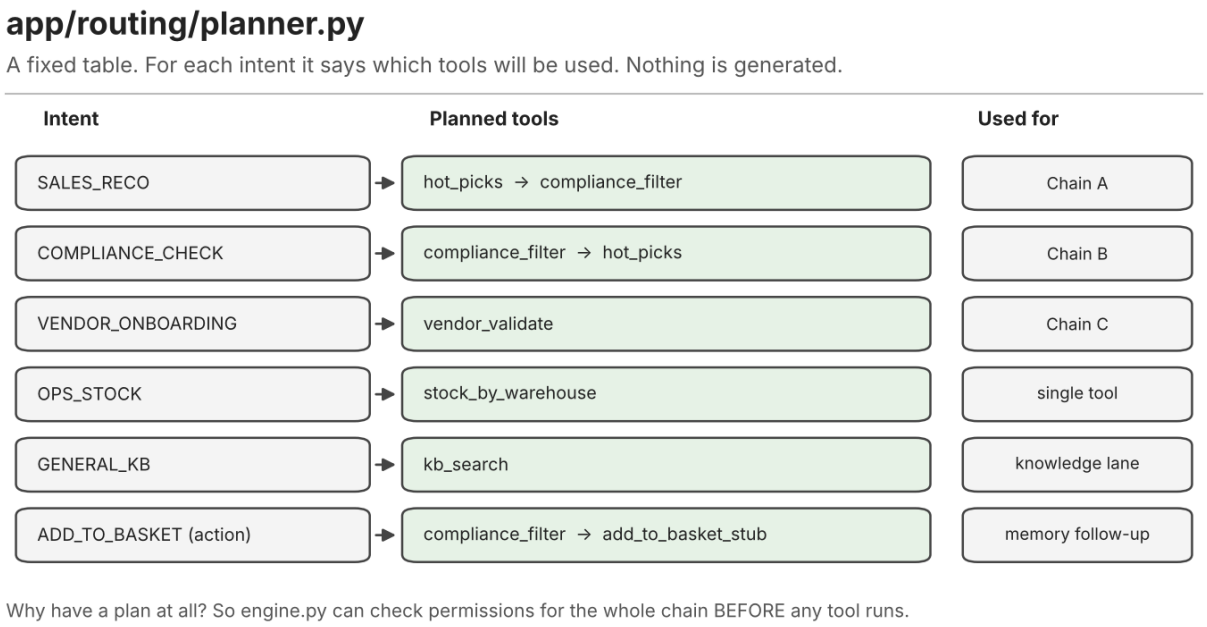
How it works

- The query is lowercased and tested against five regex rules, in order. The first rule that matches wins, so exactly one intent is chosen.
- Order: (1) add ... basket/cart, (2) vendor words like upload, lab report, net wt, (3) stock words, (4) compliance words like blocked, legal, allowed, why is, (5) sales words like hot picks, recommend, top N, under $N.
- If nothing matches, the intent is GENERAL_KB and the lane is KNOWLEDGE. Every other intent uses the TRUTH lane.
- Rule 1 is a special case. "Add 2 of the first one to the basket" is still labelled SALES_RECO, but it carries the action ADD_TO_BASKET so the planner picks a different chain.

Why I did it this way

Regex costs nothing, answers instantly, and gives the same result every time, which makes it easy to unit-test. The order matters: a message like "Why is SKU-1012 not available in CA? Suggest alternatives." contains the word "suggest" (a sales word) but also "why is" (a compliance phrase). Compliance is checked first, so it is routed correctly. The trade-off is that odd phrasing can slip through. In production I would add a small classifier as a fallback and measure it on a set of real questions.

app/routing/planner.py



What it does

Says which tools an intent will use. It is just a lookup table.

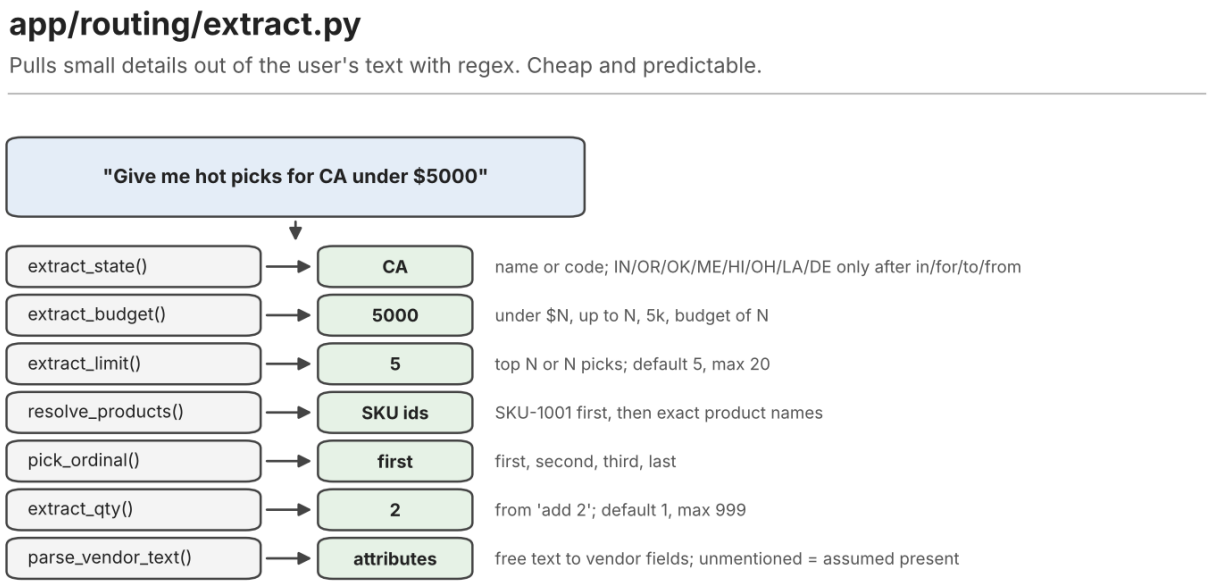
How it works

- CHAINS maps each intent to a list of tool names, for example SALES_RECO becomes hot_picks then compliance_filter.
- ACTION_CHAINS handles the basket action: compliance_filter then add_to_basket_stub.
- make_plan() returns a small Plan object with the intent, action, lane and tool list.

Why I did it this way

The plan exists so engine.py can check permissions for the whole chain *before* running anything. It is fixed, not generated by a model, so it can never invent a tool.

app/routing/extract.py



What it does

Pulls small details out of the user's sentence: the state, the budget, how many results, which product, which item in an earlier list, and the quantity.

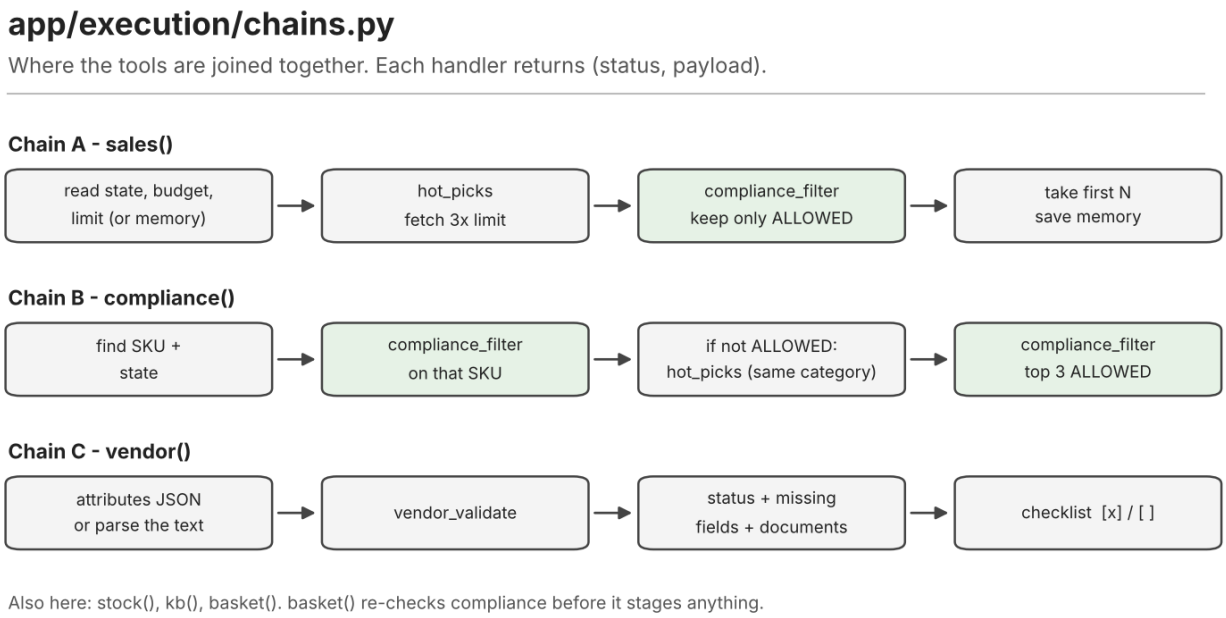
How it works

- **State:** accepts a state name ("California") or a code ("CA"). Codes that are also normal English words (IN, OR, OK, ME, HI, OH, LA, DE) are only accepted right after in, for, to or from.
- **Budget:** understands "under $5000", "up to 5,000", "budget of 5k" and a plain "$5000".
- **Limit:** "top 3" or "5 picks". The default is 5 and the maximum is 20.
- **Product:** looks for SKU-1234 first, then for an exact product name. Unknown SKUs are returned separately so the chain can say they were not found.
- **Ordinal and quantity:** "first", "second", "last", and "add 2".
- **Vendor text:** turns a sentence like "missing Net Wt and no lab report" into a fields dictionary. Anything the user did not mention is *assumed present*, and the answer says so.

Why I did it this way

Again, regex is cheap and predictable. The ambiguous state codes were a real problem: "Is it available in OR?" is fine, but "in or out of stock" would otherwise be read as Oregon. The rule of only trusting those codes after a preposition solves most of that.

app/execution/chains.py



What it does

This is where tools are joined into real workflows. Each handler returns a status and a payload, and sets the memory updates that should be saved if everything passes.

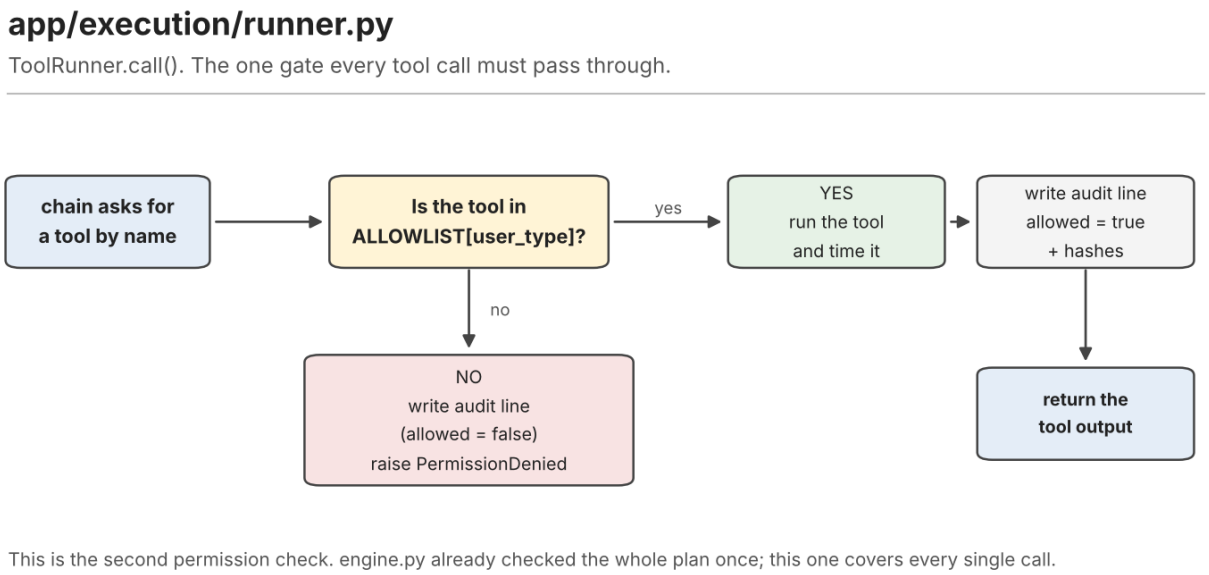
How it works

- **Chain A (sales):** read state, budget and limit from the text or from memory. Fetch 3x the limit from hot_picks, because some will be filtered out. Run compliance_filter on all of them, keep only ALLOWED, and take the first N. If no state is known it asks "Which state are you selling to?" and returns NEEDS_INFO.
- **Chain B (compliance):** find the SKU, run compliance_filter. If it is not ALLOWED, fetch popular products, remove the product itself and prefer the same category, run them through compliance_filter again, and keep the top 3 ALLOWED as alternatives.
- **Chain C (vendor):** use the attributes JSON if the caller sent one, otherwise parse the sentence. vendor_validate returns the status, missing fields and missing documents.
- **stock(), kb(), basket():** the stock flow calls stock_by_warehouse. kb() calls kb_search with the visibilities this user may see. basket() reads "first" from memory, re-checks compliance, and only then calls the stub.

Why I did it this way

Chains are the only place that knows the business order of operations, so the tools can stay small and single-purpose. The most important rule here is that alternatives and basket items always go through compliance_filter again. Nothing is trusted just because an earlier step produced it.

app/execution/runner.py



What it does

ToolRunner is the one gate that every tool call must pass through.

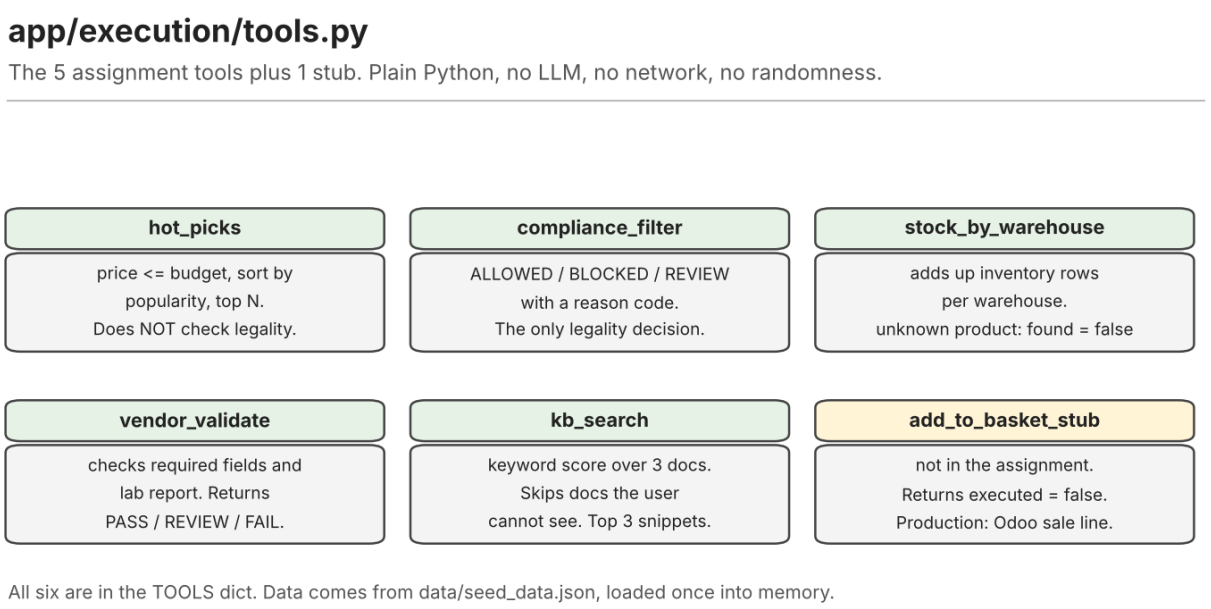
How it works

- It checks that the tool name is in the allowlist for this user type. If not, it records the attempt in the trace, writes an audit line with allowed = false, and raises PermissionDenied.
- If allowed, it runs the tool inside trace.tool(), which measures the latency and stores a hash of the output.
- It then writes one audit line (who, which tool, allowed = true, hash of the arguments, hash of the output) and returns the output.

Why I did it this way

Checking permissions in engine.py is not enough on its own. If somebody later adds a new chain and forgets the plan check, the runner still stops a forbidden call. Two checks, one at plan time and one at call time, is cheap insurance. It also means there is exactly one place to look when you audit what tools were used.

app/execution/tools.py



What it does

The business logic. Six plain Python functions: the five tools from the assignment plus a basket stub. There is no LLM, no network call and no randomness anywhere in this file.

How it works

- **hot_picks:** keeps products whose price is within the budget, sorts by popularity (ties broken by product_id so the order is always the same), and returns the top N. It does not check legality on purpose.
- **compliance_filter:** gives each product a verdict (ALLOWED, BLOCKED or REVIEW) and a reason code. It is the only place a compliance decision is made.
- **stock_by_warehouse:** adds up the inventory rows for a product. An unknown product returns found = false.
- **vendor_validate:** needs name, category, net_wt_oz and net_vol_ml, plus nicotine_mg for nicotine products. A lab report is needed for THC, CBD, kratom and mushroom. Returns PASS, REVIEW or FAIL with a checklist.
- **kb_search:** keyword matching. A title match counts double. It skips documents the user is not allowed to see and returns the top 3 snippets.
- **add_to_basket_stub:** not part of the assignment. It exists for the follow-up demo and returns executed = false. In production it would create an Odoo sale order line.
- The seed file is loaded once. If it has trailing commas from hand editing, the loader cleans them and still reads it.

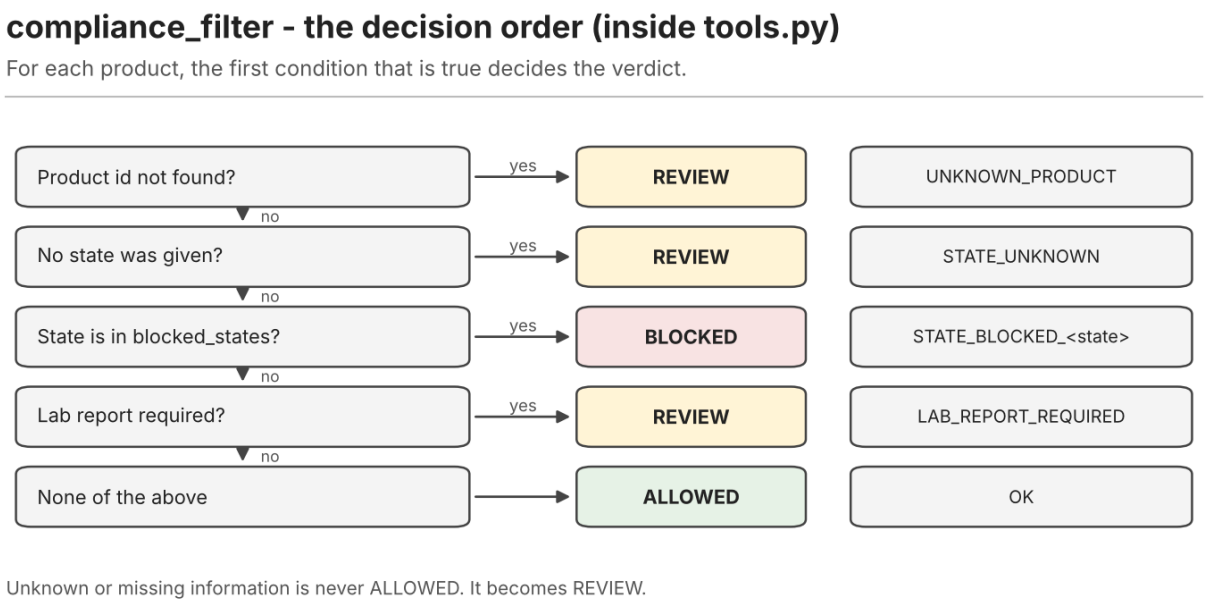


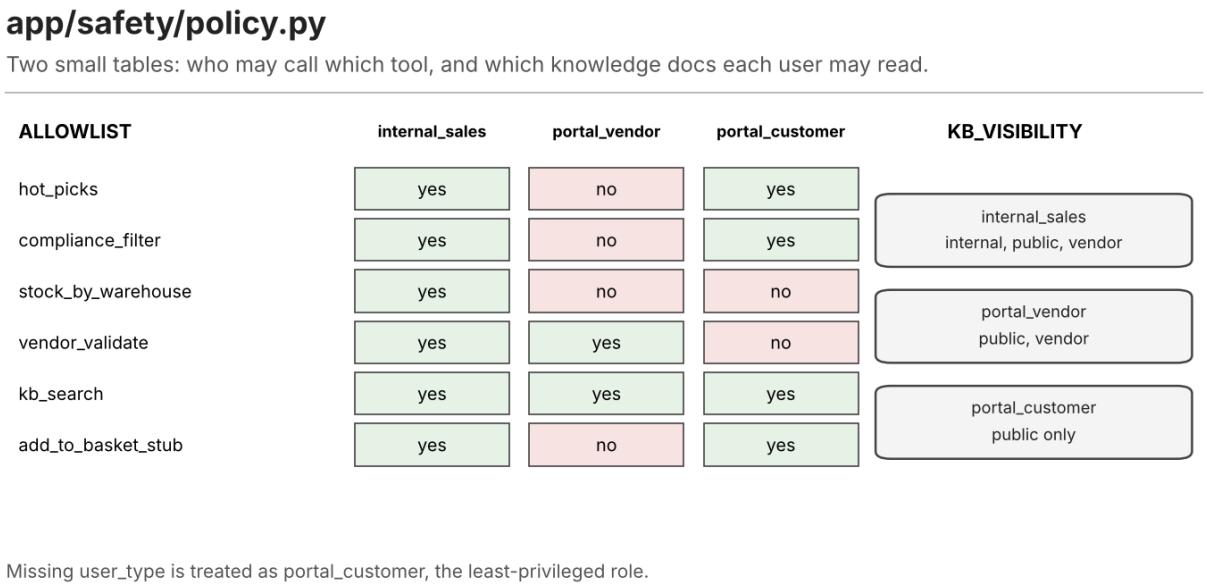
Figure. The order of checks inside compliance_filter. The first true condition decides.

Reason codes you will see in answers: **STATE_BLOCKED_CA** (the state is on the product's blocked list), **LAB_REPORT_REQUIRED** (a verified lab report is needed, so REVIEW), **STATE_UNKNOWN** and **UNKNOWN_PRODUCT** (not enough information, so REVIEW and never ALLOWED).

Why I did it this way

Each tool does one job, so each one can be tested alone and later replaced by an Odoo or REST call with the same inputs and outputs. Keeping hot_picks and compliance_filter separate means ranking can never accidentally decide legality, and the legal rules live in exactly one function.

app/safety/policy.py



What it does

Two small dictionaries: ALLOWLIST (which tools each user type may call) and KB_VISIBILITY (which knowledge documents each user type may read).

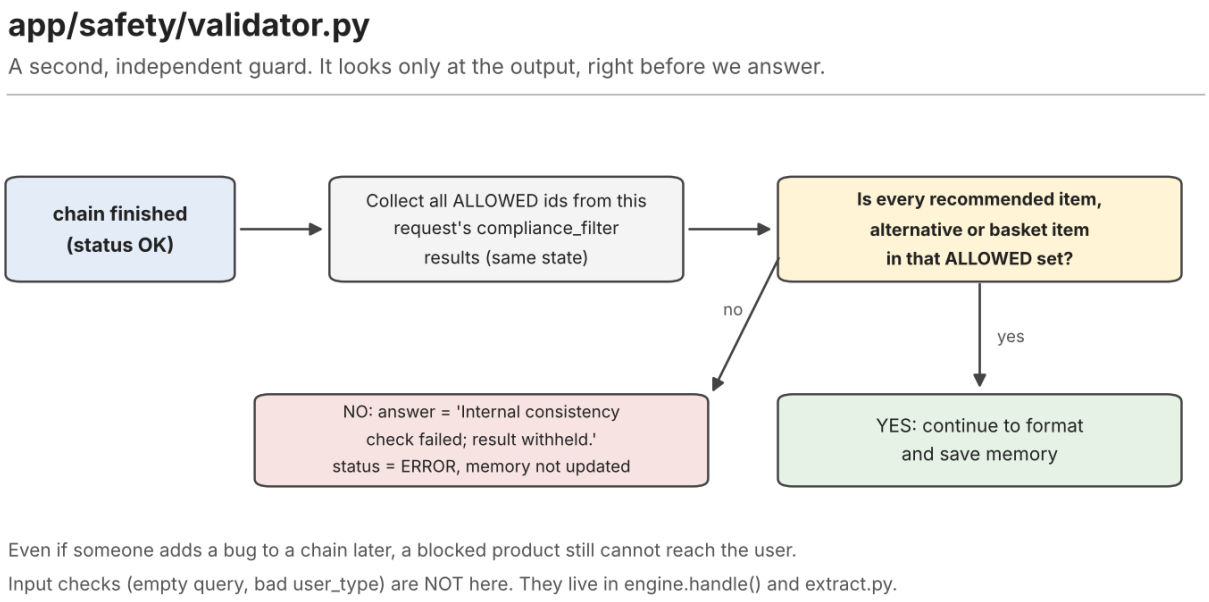
How it works

- internal_sales can use all six tools. portal_vendor can only use vendor_validate and kb_search. portal_customer can use hot_picks, compliance_filter, kb_search and the basket stub, but not stock or vendor tools.
- internal_sales sees internal, public and vendor documents. portal_vendor sees public and vendor. portal_customer sees public only.
- It is used in three places: engine.py at plan time, runner.py at call time, and chains.py for the knowledge search.

Why I did it this way

Permissions are data, not code scattered across files. If the rules change, one file changes. An unknown or missing role is treated as the least-privileged one (portal_customer) so a mistake fails closed.

app/safety/validator.py



What it does

A second, independent guard that looks at the output just before we answer.

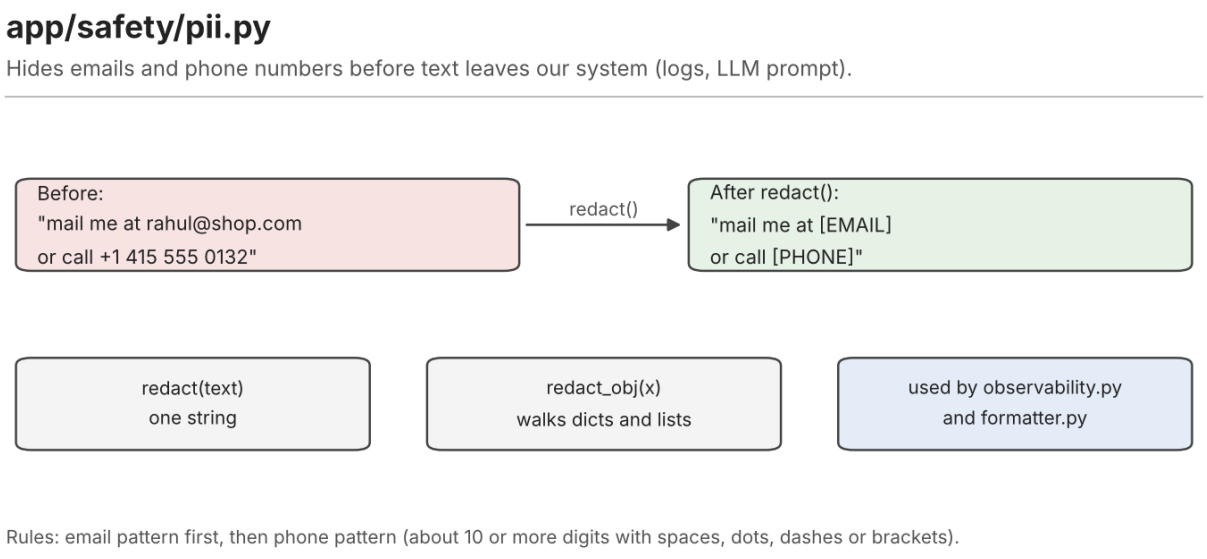
How it works

- It collects every product id that the compliance_filter marked ALLOWED *in this same request* for the same state.
- Then it checks that every recommended product, every alternative and the basket item is in that set.
- If anything is missing, the answer becomes "Internal consistency check failed; result withheld.", the status is ERROR, memory is not saved, and the problem is written to validation_failures in the trace.

Why I did it this way

A rule that only lives in the chain is only as good as the last person who edited the chain. The validator is a safety net that does not trust the chain. Even if a bug is added later, a blocked product still cannot reach the user. It only checks outputs. Input checks live in engine.handle() and extract.py.

app/safety/pii.py



What it does

Hides emails and phone numbers before text leaves our system.

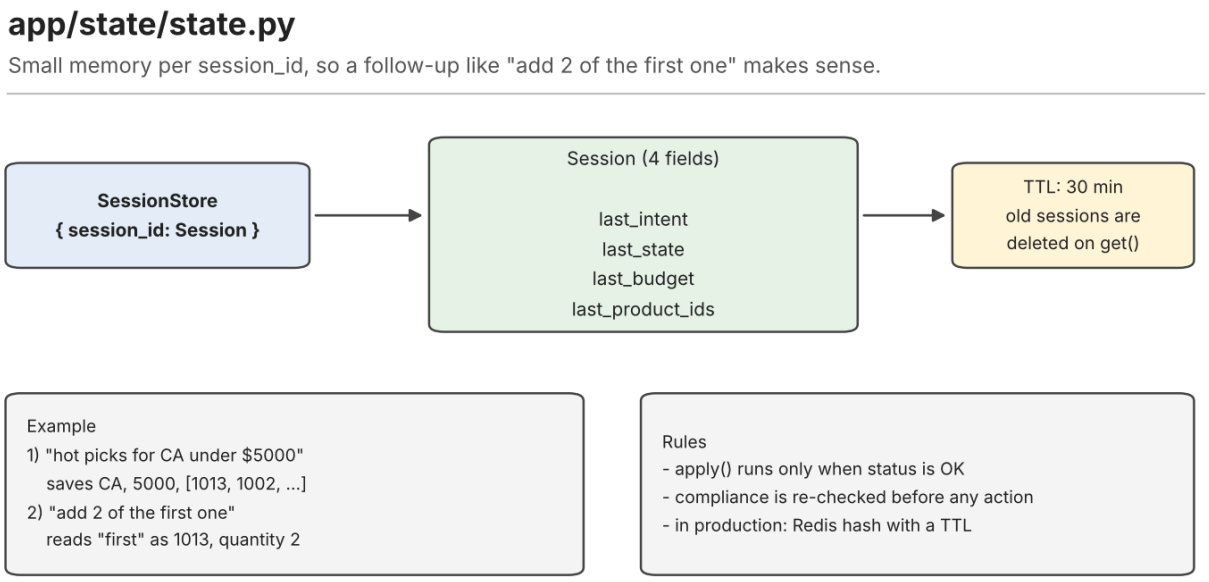
How it works

- redact(text) replaces emails with [EMAIL] and phone-like numbers with [PHONE].
- redact_obj(x) does the same inside dictionaries and lists, so a whole payload can be cleaned in one call.
- It is used when the query is stored in the trace and when the prompt for a possible LLM is built.

Why I did it this way

Logs and prompts are the places where customer data usually leaks by accident. Cleaning it at the edge, in one small function, is simpler than hoping every caller remembers to do it. A pattern like this will miss unusual formats, so in production I would add a proper PII library.

app/state/state.py



What it does

Short-term memory for a conversation, kept per session_id.

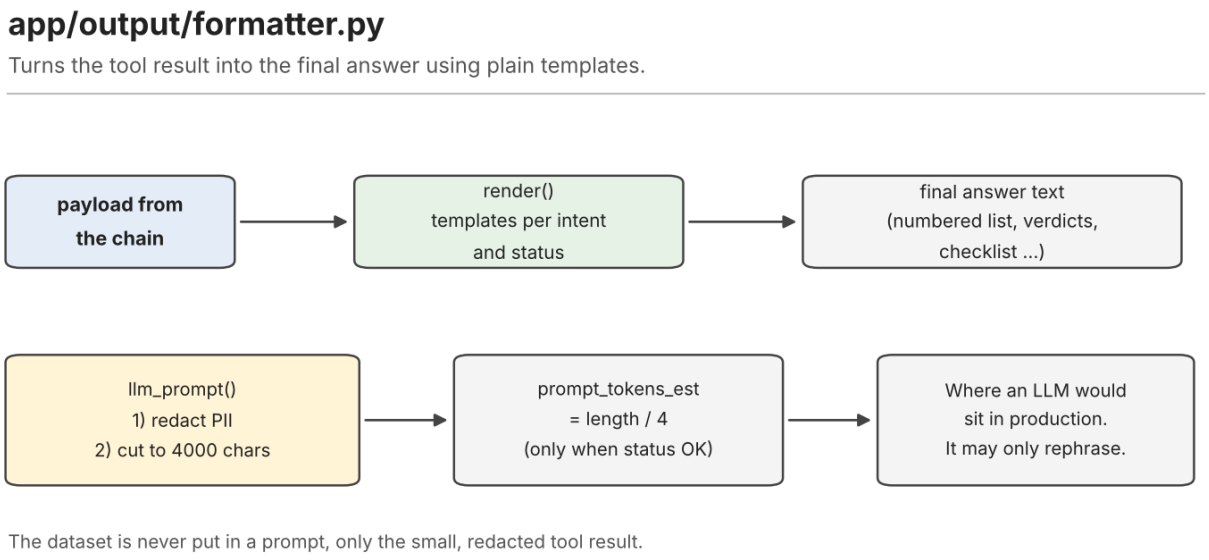
How it works

- Each session stores four fields: last_intent, last_state, last_budget and last_product_ids.
- Example: after "hot picks for CA under $5000", the system remembers CA, 5000 and the five product ids. Then "Ok add 2 of the first one to the basket" reads "first" as product 1013 and quantity 2.
- Memory is only written after the validator passes, and compliance is re-checked before the basket action, so memory can never override a rule.
- If you say "add 2" with no earlier list, it asks you to ask for hot picks first. If there are several items and you do not say which, it asks "Which one?".
- Sessions expire after 30 minutes. The store removes old ones every time it is read.

Why I did it this way

Four fields is enough for the follow-ups in the assignment and nothing more. A bigger memory would just be more state to get wrong. It is a plain dictionary here; in production it would be a Redis hash with a time limit, so every server shares it.

app/output/formatter.py



What it does

Turns the tool result into the final text the user reads.

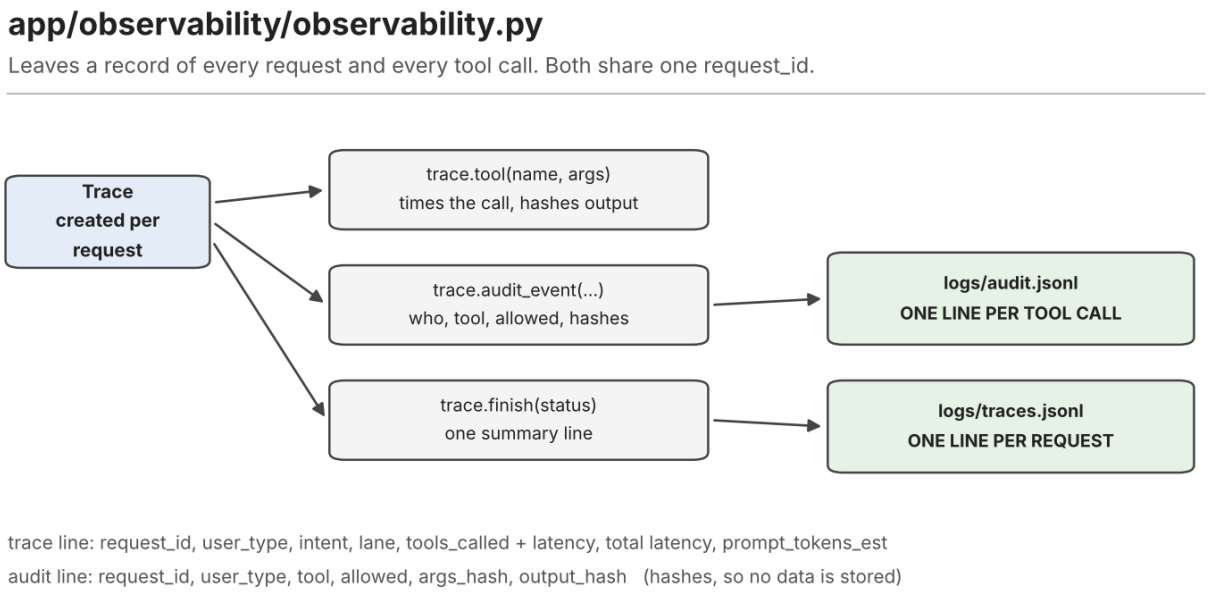
How it works

- render() has one template per intent and one for each message-type status. For sales it prints a numbered list plus a line like "Excluded by compliance: 4 blocked, 2 need review."
- llm_prompt() shows where a model would be used in production. It redacts PII and cuts the tool result at 4000 characters. Only that small piece is ever sent, never the dataset.
- prompt_tokens_est is the length of that prompt divided by 4, a rough token count. It is only calculated when the status is OK.

Why I did it this way

Templates make the answer predictable and safe. I added llm_prompt() so the size and privacy limits are visible and measurable now, even though no LLM is called yet. When one is added, it only rephrases what is already decided.

app/observability/observability.py



What it does

Keeps a record of what happened, for debugging and for audit.

How it works

- Every request creates a Trace with a short request_id. The same id is written to both log files.
- **logs/traces.jsonl** gets one line per request: time, request_id, session, user type, intent, lane, status, the redacted query, tools called with their latency, denied tools, validation failures, total latency and prompt_tokens_est.
- **logs/audit.jsonl** gets one line per tool call: who called it, which tool, allowed or not, a hash of the arguments and a hash of the output.
- The audit file stores hashes instead of data, so it proves what happened without keeping the data. Set LOG_DIR to change where logs go, and LOG_STDERR=0 to stop the trace being printed.

Why I did it this way

If a salesperson asks why a product was not recommended, I want to answer from evidence, not memory. Using the same request_id in both files makes it easy to join them. In production the same shapes could go to an append-only store or a tool like Langfuse.

8. A full example, step by step

User type: internal_sales. Query: "Give me hot picks for CA under $5000". This is the real output from running main.py.

- **Entry:** engine.handle() checks the user type and the query.
- **Route:** the sales rule matches. Intent = SALES_RECO, lane = TRUTH.
- **Plan:** hot_picks, then compliance_filter (Chain A).
- **Authorize:** internal_sales may use both tools, so it passes.

- **Execute:** state = CA, budget = 5000, limit = 5. hot_picks returns 15 candidates (3 x 5). compliance_filter marks 4 BLOCKED, 2 REVIEW and 9 ALLOWED. The first 5 allowed ones are kept.
- **Validate:** all 5 have an ALLOWED verdict.
- **Format:** a numbered list plus the excluded counts.
- **Persist:** memory saves CA, 5000 and the 5 product ids. One trace line and two audit lines are written.

```
Top picks for CA under $5,000.00:
1. SKU-1013 - Accessories - Ember 86 - $256.69 (popularity 0.955)
2. SKU-1002 - Nicotine Vape - Cloud 433 - $229.66 (popularity 0.947)
3. SKU-1053 - Accessories - Ember 291 - $94.25 (popularity 0.906)
4. SKU-1027 - Mushroom Chocolate - Ember 122 - $44.90 (popularity 0.885)
5. SKU-1005 - THC Beverage - Haven 51 - $213.88 (popularity 0.875)
Excluded by compliance: 4 blocked, 2 need review.
```

The other five demo prompts

User and prompt	Intent / status	What came back
internal_sales: "Ok add 2 of the first one to the basket"	SALES_RECO (basket action) / OK	Staged 2 x SKU-1013. Compliance re-checked for CA: ALLOWED. Basket write is a stub.
internal_sales: "Why is SKU-1012 not available in CA? Suggest alternatives."	COMPLIANCE_CHECK / OK	BLOCKED, reason STATE_BLOCKED_CA. Allowed alternatives: SKU-1027, SKU-1020, SKU-1045 (all Mushroom Chocolate).
internal_sales: "How much stock does SKU-1001 have and where?"	OPS_STOCK / OK	Total 471: NY 222, LA 214, TX 35.
portal_vendor: "I'm uploading a product missing Net Wt and no lab report - what do I fix?"	VENDOR_ONBOARDING / OK	Status FAIL. Fix net_wt_oz. Upload a Lab Report. Notes that unmentioned fields were assumed present.
portal_customer: "How much stock does SKU-1001 have?"	OPS_STOCK / DENIED	Permission denied: portal_customer cannot use stock_by_warehouse. No tool ran.

9. What the response looks like

Field	Meaning
request_id	Unique id. The same id is in traces.jsonl and audit.jsonl.
intent	The one intent the router chose.
status	OK, NEEDS_INFO, DENIED, BLOCKED_ACTION, ERROR or INVALID_INPUT.
answer	The final text, built only from templates and tool results.
data	The structured result behind the answer, so a UI could use it.
session	The 4 memory fields after this request.
meta	Planned chain, tools called with latency, total latency, and the prompt token estimate.

Status	When it happens
OK	Tools ran and the validator passed.

Status	When it happens
NEEDS_INFO	Something is missing, for example no state. The system asks a question back.
DENIED	The user type cannot use a tool in the plan.
BLOCKED_ACTION	A follow-up action is on a product that is no longer ALLOWED.
ERROR	The validator found an item without ALLOWED. This is the safety net and should never happen.
INVALID_INPUT	Empty query or unknown user_type.

10. What a log line looks like

This is the trace line for the first demo request (shortened a little). The audit file has two matching lines for it, one for hot_picks and one for compliance_filter, with the same request_id.

```
{"request_id": "edd5fad36f59", "session_id": "s1", "user_type": "internal_sales",
 "intent": "SALES_RECO", "lane": "TRUTH", "status": "OK",
 "query": "Give me hot picks for CA under $5000",
 "tools_called": [
   {"tool": "hot_picks", "args": {"state": "CA", "budget": 5000.0, "limit": 15},
    "latency_ms": 0.459, "output_hash": "6349441082a0ad38"},
   {"tool": "compliance_filter", "args": {"state": "CA", "product_ids": [1056, 1013, ...]},
    "latency_ms": 0.07, "output_hash": "8f9029b58d99130c"}],
 "denied_tools": [], "validation_failures": [],
 "latency_ms_total": 22.946, "prompt_tokens_est": 258}
```

prompt_tokens_est is the size of the redacted, capped tool result divided by 4. It is 0 when nothing is formatted, for example on a DENIED request.

11. How to run it

```
python main.py                      # runs the 6 demo prompts
python main.py chat                 # interactive, as internal_sales
python main.py chat portal_vendor   # interactive, as another user type
```

Optional environment variables: SEED_PATH (use another data file), LOG_DIR (where logs are written), LOG_STDERR=0 (do not print trace lines). Type hints like `str | None` need Python 3.10 or newer. On older versions they would need to be changed to Optional[str].

12. What I did not use, and why

- **No LLM for routing:** regex is free and instant.
- **No LLM for facts:** a model can invent stock or legality.
- **No embeddings or RAG:** not appropriate for live inventory or compliance.
- **No database:** the assignment gives a JSON file, so the tools load it once into memory.
- **No LangChain or LangGraph:** the pipeline is 7 plain steps. It is easier to read and to test without a framework.

I kept it light on purpose. The point of this PoC is system design, not frameworks. Each file has one job, so tools.py can later turn into real Odoo calls and state.py can turn into Redis without touching the rest.

13. Honest limits, and what I would change for production

- **Routing is regex.** Unusual wording can be misrouted. I would add a small classifier as a fallback and test it on real questions.
- **Vendor text is assumed complete.** Fields the user does not mention are treated as present, and the answer says so. The full attributes JSON gives a proper check.
- **Over-fetching is a fixed 3x.** If more than two thirds of the candidates are blocked, fewer than N picks come back.
- **One state per request.** The sales chain works with a single destination state.
- **Memory is in-process.** A restart clears it. In production it would be Redis with a time limit.
- **The basket is a stub** and does not write to any system. It would become an Odoo sale order line.
- **Knowledge search is keyword matching** over 3 documents. It is fine for a PoC but not for a large document set.
- **PII redaction is pattern-based** and only covers emails and phone numbers.
- **customers and vendors** exist in the data file but no tool reads them yet.

14. Questions you might be asked

Q. Why not just ask an LLM?

Because stock, price and legality must be exactly right and repeatable. An LLM can sound confident and still be wrong. Here, plain functions decide the facts, and a model (if added) can only rephrase them.

Q. So where would the LLM go?

In formatter.py. It would get only the small, redacted, size-capped tool result from llm_prompt() and could only reword it. It would never see the dataset and never make a decision.

Q. Why is routing done with regex?

It costs nothing, is instant and always gives the same answer, so it is easy to test. The weakness is unusual phrasing. In production I would add a small classifier as a fallback and check it on a set of real questions.

Q. How do you make sure a blocked product is never recommended?

Three layers. compliance_filter is the only place that decides legality. The chains always run candidates and alternatives through it. Then the validator checks again that everything in the answer had an ALLOWED verdict, and the basket action re-checks compliance before staging anything.

Q. What happens if user_type is missing?

It becomes portal_customer, the least-privileged role. An unknown user_type is rejected as INVALID_INPUT.

Q. Why doesn't hot_picks check legality?

So each tool has one job. Ranking and legality are separate questions. If hot_picks also decided legality there would be two places with legal rules.

Q. Why does hot_picks fetch 3 times the limit?

Because some products will be filtered out by compliance. Fetching extra means we usually still have enough allowed items left to return the number requested.

Q. What is logged, and is personal data stored?

One line per request and one line per tool call, sharing a request_id. Emails and phone numbers are redacted before logging. The audit file stores hashes of arguments and outputs, not the data itself.

Q. How would this move to real Odoo?

Each tool in tools.py becomes an Odoo or REST call with the same input and output. state.py moves to Redis. The router, chains, permissions and validator stay as they are.

Q. What if someone asks something unrelated, like the weather?

It goes to GENERAL_KB and returns "No matching document found". The system only answers inside its domain by design.

Q. Why no LangChain or a database?

The assignment is about system design. Seven plain steps are easy to read and test. The data is a JSON file, so loading it once into memory is enough.